

알고리즘 실습 기말 프로젝트

무인편의점 관리자

과 목 명 : 알고리즘 실습

담당 교수 : 차 가 을

학번 : 2601110288

이름 : 박수용

제 출 일 : 2026년 6월 22일

목차

1. 프로젝트 개요
2. 게임 기획 및 설계
 - 2.1 게임 소개
 - 2.2 게임 규칙
 - 2.3 게임 흐름 (플로우차트)
3. 구현 내용
 - 3.1 기능 목록
 - 3.2 함수 목록
 - 3.3 자료구조 설계
 - 3.4 C언어 문법 활용
4. 핵심 코드 설명
 - 4.1 Player 구조체 및 초기화
 - 4.2 Fisher-Yates 셔플 알고리즘
 - 4.3 입력 검증 함수 (getChoice)
 - 4.4 파일 입출력 (saveHighScore / loadHighScore)
5. 실행 결과
 - 5.1 메인 메뉴 화면
 - 5.2 게임 진행 화면
 - 5.3 잘못된 입력 처리 화면
 - 5.4 최종 결과 출력 화면
 - 5.5 최고 점수 확인 화면
6. 어려웠던 점 및 해결 방법
 - 6.1 Fisher-Yates 셔플 구현
 - 6.2 구조체 포인터와 화살표 연산자
 - 6.3 입력 버퍼 처리 문제
7. 결론 및 느낀 점
8. 참고 자료

1. 프로젝트 개요

본 프로젝트는 알고리즘 실습 과목의 기말 과제로, 한 학기 동안 배운 C언어 문법을 종합적으로 활용하여 직접 게임을 기획하고 구현한 결과물입니다. 게임 제목은 「무인편의점 관리자」로, 플레이어가 무인편의점의 관리자가 되어 CCTV로 매장 상황을 감시하고 다양한 돌발 상황에 대응하는 선택형 시뮬레이션 게임입니다.

아이디어의 출발점은 최근 빠르게 늘어나고 있는 무인편의점, 무인카페 등 비대면 매장 트렌드였습니다. 실제로 무인매장에서는 도난, 결제 오류, 고객 민원 등 다양한 상황이 발생할 수 있는데, 이를 게임화하면 흥미로운 경험을 제공할 수 있을 것이라 판단하였습니다. 특히 보안을 강화하면 평판이 낮아지고, 느슨하게 대응하면 도난 위험도가 높아지는 '균형' 구조를 핵심 재미 요소로 설계하였습니다.

2. 게임 기획 및 설계

2.1 게임 소개

「무인편의점 관리자」는 플레이어가 무인편의점 관리자의 역할을 맡아, 매장 CCTV 화면을 통해 발생하는 다양한 상황에 대응하는 선택형 시뮬레이션 게임입니다. 매 턴마다 랜덤으로 상황이 발생하며, 플레이어는 제시된 세 가지 선택지 중 하나를 골라 대응합니다. 선택 결과에 따라 보안 점수, 매출, 평판, 도난 위험도 네 가지 상태값이 변화합니다.

단순히 높은 점수를 얻는 게임이 아니라, 보안을 지나치게 강화하면 고객 평판이 낮아지고 반대로 느슨하게 대응하면 도난 위험도가 높아지는 트레이드오프 구조를 설계의 핵심으로 삼았습니다. 플레이어는 매 상황마다 균형 잡힌 판단을 내려야 합니다.

상태값	초기값	설명
보안 점수	50	높을수록 좋음. 보안 대응 수준을 나타냅니다.
매출	100	높을수록 좋음. 매장 수익을 나타냅니다.
평판	50	높을수록 좋음. 고객 만족도를 나타냅니다.
도난 위험도	30	낮을수록 좋음. 100 이상이면 게임 오버.

2.2 게임 규칙

게임의 기본 규칙은 다음과 같습니다.

- 게임이 시작되면 플레이어의 이름을 입력합니다.
- 플레이어는 무인편의점 관리자가 되며, 총 5턴 동안 게임이 진행됩니다.
- 매 턴마다 8개의 이벤트 중 하나가 중복 없이 랜덤으로 출력됩니다.
- 플레이어는 출력된 상황을 읽고 1, 2, 3번 중 하나를 선택합니다.
- 선택 결과에 따라 보안 점수, 매출, 평판, 도난 위험도가 변화합니다.
- 도난 위험도가 100 이상이 되거나 평판이 0 이하가 되면 게임 오버입니다.
- 5턴을 모두 버티면 최종 점수(보안+매출+평판-위험도)를 계산하고 S/A/B/C 등급을 출력합니다.
- 게임 종료 후 최고 점수를 파일(highscore.txt)에 자동 저장합니다.

최종 등급 기준은 다음과 같습니다.

등급	점수 기준	메시지
★ S등급	200점 이상	매우 우수한 무인편의점 관리자입니다!
A등급	150점 이상	안정적으로 매장을 관리하였습니다.
B등급	100점 이상	기본적인 관리는 하였지만 보완이 필요합니다.
C등급	100점 미만	매장 관리에 어려움이 있었습니다.

2.3 게임 흐름 (플로우 차트)



3. 구현 내용

3.1 주요 기능 상세 설계

기능 1

기능명	메인 메뉴 기능
동작 설명	프로그램 실행 시 메인 메뉴 화면을 출력합니다. 플레이어가 게임 시작, 게임 설명 보기, 최고 점수 확인, 종료 중 하나를 선택할 수 있도록 구성하였습니다. 선택에 따라 해당 기능이 실행됩니다.
입력 / 출력	입력: 메뉴 번호(1~4) / 출력: 선택된 메뉴에 해당하는 화면
관련 함수명 (예정)	showMenu()

기능 2

기능명	플레이어 정보 입력 및 초기화 기능
동작 설명	게임 시작 시 플레이어의 이름을 입력받고, 보안 점수(50), 매출(100), 평판(50), 도난 위험도(30)를 초기값으로 설정합니다. 구조체 포인터를 활용하여 값을 직접 변경합니다.
입력 / 출력	입력: 플레이어 이름(문자열) / 출력: 초기화된 Player 구조체
관련 함수명 (예정)	initPlayer(Player *p)

기능 3

기능명	랜덤 상황 발생 기능
동작 설명	총 8개의 이벤트 중 5개를 중복 없이 랜덤으로 선택하여 출력합니다. Fisher-Yates 셔플 알고리즘을 사용하여 배열을 섞은 뒤, 앞에서 5개만 사용하는 방식으로 구현하였습니다.
입력 / 출력	입력: 이벤트 번호 배열 / 출력: 섞인 순서의 이벤트 배열
관련 함수명 (예정)	shuffleEvents(int arr[], int size)

기능 4

기능명	선택지 입력 및 처리 기능
동작 설명	각 상황에 대해 1, 2, 3번 선택지를 제시하고, 플레이어의 입력을 받아 처리합니다. 잘못된 입력(문자, 범위 초과)이 들어올 경우 입력 버퍼를 비우고 다시 입력받도록 구현하였습니다.
입력 / 출력	입력: 선택 번호(1~3) / 출력: 유효한 선택 번호 반환
관련 함수명 (예정)	getChoice()

기능 5

기능명	상태값 관리 및 이벤트 처리 기능
동작 설명	플레이어의 선택에 따라 보안 점수, 매출, 평판, 도난 위험도를 증가 또는 감소시킵니다. switch-case 문을 사용하여 8개 이벤트를 각각 처리하며, 구조체 포인터를 통해 값을 직접 변경합니다.
입력 / 출력	입력: Player 구조체 포인터, 이벤트 번호 / 출력: 변경된 상태값
관련 함수명 (예정)	handleEvent(Player *p, int eventNum)

기능 6

기능명	현재 상태 출력 기능
동작 설명	매 턴이 끝날 때마다 현재 보안 점수, 매출, 평판, 도난 위험도를 화면에 출력합니다. 플레이어가 자신의 상태를 확인하고 다음 선택에 참고할 수 있도록 합니다.
입력 / 출력	입력: Player 구조체(값 전달) / 출력: 상태값 콘솔 출력
관련 함수명 (예정)	showStatus(Player p)

기능 7

기능명	게임 종료 판정 및 결과 출력 기능
동작 설명	도난 위험도가 100 이상이거나 평판이 0 이하가 되면 게임이 실패로 종료됩니다. 5턴을 모두 버틴 경우 최종 점수를 계산하고, S/A/B/C 등급을 출력합니다.
입력 / 출력	입력: Player 구조체(값 전달) / 출력: 등급 및 최종 점수 콘솔 출력
관련 함수명 (예정)	showResult(Player p)

기능 8

기능명	최고 점수 저장 및 불러오기 기능
동작 설명	게임 종료 시 최종 점수가 기존 최고 점수보다 높으면 파일(highscore.txt)에 저장합니다. 파일이 없는 경우에도 오류 없이 동작하도록 fopen 반환값을 확인하는 방식으로 처리하였습니다.
입력 / 출력	입력: Player 구조체 / 출력: highsocre.txt 파일 생성 또는 갱신
관련 함수명 (예정)	saveHighScore(Player p), loadHighScore()

3.2 함수 목록 설계

함수명	반환형	매개변수	역할 설명
showMenu	void	없음	메인 메뉴 화면을 출력합니다.
showGameInfo	void	없음	게임 설명(규칙, 목표)을 출력합니다.
initPlayer	void	Player *p	플레이어 이름 입력 및 상태값 초기화를 수행합니다.
showStatus	void	Player p	현재 보안, 매출, 평판, 위험도를 출력합니다.
playGame	void	Player *p	게임 전체 흐름을 제어합니다. (셔플 → 턴 반복 → 종료 판정)
handleEvent	void	Player *p, int eventNum	이벤트 번호에 따른 상황 출력 및 선택 결과를 처리합니다.
getChoice	int	없음	1~3 범위의 유효한 입력을 받아 반환합니다. 잘못된 입력 시 재입력을 요청합니다.
shuffleEvents	void	int arr[], int size	Fisher-Yates 셔플로 이벤트 배열을 랜덤하게 섞습니다.
showResult	void	Player p	최종 점수 계산 및 S/A/B/C 등급을 출력합니다.
saveHighScore	void	Player p	최고 점수를 파일(highscore.txt)에 저장합니다.
loadHighScore	void	없음	파일에서 최고 점수를 읽어 출력합니다.

3.3 자료구조 설계

구조체 정의

구조체명	멤버 변수	자료형	역할 설명
Player	name	char[30]	플레이어의 이름을 저장합니다.
	security	int	보안 점수를 저장합니다. (초기값: 50)
	sales	int	매출을 저장합니다. (초기값: 100)
	reputation	int	평판을 저장합니다. (초기값: 50)
	risk	int	도난 위험도를 저장합니다. (초기값: 30)
	score	int	최종 점수를 저장합니다.

사용할 전역 변수 / 배열

자료형	변수명	역할 설명
int	order[TOTAL_EVENTS]	이벤트 순서를 저장하는 배열입니다. {0,1,2,...,7}로 초기화한 뒤 셔플합니다.
#define	MAX_TURNS (5)	한 게임당 진행할 최대 턴 수를 정의합니다.
#define	TOTAL_EVENTS (8)	전체 이벤트 개수를 정의합니다.
#define	RISK_LIMIT (100)	도난 위험도 실패 기준값을 정의합니다.
#define	REP_LIMIT (0)	평판 실패 기준값을 정의합니다.
#define	SAVE_FILE	최고 점수 저장 파일명("highscore.txt")을 정의합니다.

3.4 C 언어 문법 활용 계획 (구체적)

C 언어 개념	구체적인 활용 방법 (어느 기능에, 어떻게 적용하는지)
배열 (Array)	이벤트 순서를 저장하는 order[TOTAL_EVENTS] 배열을 선언하여 {0,1,2,...,7}로 초기화하고, 셔플 후 앞에서 5개를 꺼내 사용하였습니다. 배열을 사용함으로써 8개의 이벤트를 인덱스로 관리할 수 있었습니다.
함수 (Function)	프로그램 기능을 11개의 함수로 분리하였습니다. showMenu(), playGame(), handleEvent() 등 각 기능별로 함수를 나누어 코드의 가독성과 유지보수성을 높였습니다.
구조체 (struct)	플레이어의 이름, 보안 점수, 매출, 평판, 도난 위험도, 최종 점수를 하나의 Player 구조체로 묶어 관리하였습니다. typedef를 사용하여 구조체 이름을 간결하게 사용할 수 있도록 하였습니다.
포인터 (Pointer)	initPlayer(Player *p), playGame(Player *p), handleEvent(Player *p, int eventNum) 함수에서 구조체 포인터를 사용하여 함수 내부에서 원본 데이터를 직접 변경할 수 있도록 하였습니다. 화살표 연산자(->)를 활용하였습니다.
파일 입출력	fopen(), fprintf(), fscanf(), fclose()를 사용하여 최고 점수를 highscore.txt 파일에 저장하고, 다음 실행 시 불러오도록 구현하였습니다. 파일이 존재하지 않을 때는 "기록 없음"을 출력하도록 예외 처리하였습니다.
반복문 (for/while)	for문으로 게임 턴을 반복(5회)하고, while문으로 잘못된 입력 시 재입력을 받도록 구현하였습니다. Fisher-Yates 셔플에서도 for문을 활용하였습니다.
조건문 (if/switch)	if문으로 게임 종료 조건(위험도 100 이상, 평판 0 이하)을 판단하고, switch-case문으로 8개 이벤트를 각각 분기 처리하였습니다. 메인 메뉴 선택에서도 switch를 사용하였습니다.
문자열 (string.h)	플레이어 이름 저장에 char 배열과 scanf를 사용하였습니다. 출력 시 %s 서식 지정자를 활용하여 이름을 포함한 메시지를 출력하였습니다.
매크로 (#define)	MAX_TURNS, TOTAL_EVENTS, RISK_LIMIT, REP_LIMIT, SAVE_FILE 등 상수를 매크로로 정의하여 코드 수정이 용이하도록 하였습니다.
랜덤 함수 (rand)	srand(time(NULL))로 시드를 설정하고, rand() % size를 사용하여 이벤트 배열을 셔플하였습니다. 매 실행마다 다른 이벤트 순서가 나오도록 구현하였습니다.

4. 핵심 코드 설명

본 프로젝트에서 구현한 핵심 코드 4가지를 선정하여 동작 원리와 함께 설명합니다.

4.1 Player 구조체 및 초기화

플레이어의 모든 상태값을 하나의 구조체로 묶어 관리하였습니다. typedef를 활용하여 매번 struct 키워드 없이 Player로 간결하게 선언할 수 있도록 하였습니다.

```
typedef struct {
    char name[30];      /* 플레이어 이름(최대 30글자) */
    int security;        /* 보안 점수 */
    int sales;           /* 매출 */
    int reputation;       /* 평판 */
    int risk;            /* 도난 위험도 */
    int score;           /* 최종 점수 */
} Player;

void initPlayer(Player *p) {
    printf("\n 플레이어 이름을 입력하세요: ");
    scanf("%s", p->name);

    /* 초기 상태값 설정 */
    p->security = 50;      /* 보안 점수 초기값 */
    p->sales = 100;        /* 매출 초기값 */
    p->reputation = 50;    /* 평판 초기값 */
    p->risk = 30;          /* 도난 위험도 초기값 */
    p->score = 0;          /* 총점 초기값 */

    printf("\n %s 관리자님, 근무를 시작합니다!\n", p->name);
    printf(" 총 %d개의 상황을 해결해야 합니다.\n", MAX_TURNS);
}
```

initPlayer() 함수는 구조체 포인터(Player *p)를 매개변수로 받아, 화살표 연산자(->)를 통해 원본 구조체의 값을 직접 변경합니다. 값 복사 방식이 아닌 포인터를 사용함으로써 함수 호출 후에도 변경 사항이 main()의 player 변수에 그대로 반영됩니다.

4.2 Fisher-Yates 셔플 알고리즘

8개의 이벤트 중 5개를 중복 없이 랜덤으로 출력하기 위해 Fisher-Yates 셔플 알고리즘을 사용하였습니다. 이 알고리즘은 배열의 뒤에서부터 앞쪽 임의의 위치와 원소를 교환하는 방식으로, 모든 순열이 동일한 확률로 생성된다는 것이 수학적으로 증명된 알고리즘입니다.

```

void shuffleEvents(int arr[], int size) {
    int i, j, temp;

    /* 배열을 0, 1, 2, ... 순서로 초기화 */
    for (i = 0; i < size; i++) {
        arr[i] = i;
    }

    /* Fisher-Yates 셔플: 뒤에서부터 랜덤 위치와 교환 */
    for (i = size - 1; i > 0; i--) {
        j = rand() % (i + 1);
        temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}

```

playGame() 함수에서 shuffleEvents()를 호출하여 이벤트 순서를 섞은 뒤, 앞에서부터 MAX_TURNS(5)개만 꺼내 사용합니다. 이를 통해 매 게임마다 다른 순서의 이벤트가 등장하며 반복 플레이 시에도 신선함을 유지할 수 있습니다.

4.3 입력 검증 함수 (getChoice)

사용자가 숫자가 아닌 문자를 입력하거나 범위를 벗어난 값을 입력했을 때, 프로그램이 비정상 종료되지 않도록 입력 검증 로직을 구현하였습니다.

```

int getChoice() {
    int choice;
    int result;

    while (1) {
        printf(" 선택 (1/2/3): ");
        result = scanf("%d", &choice);

        /* scanf가 숫자를 읽지 못한 경우 (문자 입력 등) */
        if (result != 1) {
            /* 입력 버퍼를 비웁니다 */
            while (getchar() != '\n');
            printf(" [오류] 숫자 1, 2, 3 중 하나를 입력해 주세요.\n");
            continue;
        }

        /* 1~3 범위 확인 */
        if (choice >= 1 && choice <= 3) {
            return choice; /* 정상 입력이면 반환 */
        }

        printf(" [오류] 1, 2, 3 중 하나를 선택해 주세요.\n");
    }
}

```

scanf()의 반환값이 1이 아닌 경우(숫자가 아닌 문자 입력 시)를 별도로 처리하고, getchar()로 입력 버퍼를 비워 다음 입력에 영향을 주지 않도록 하였습니다. 이를 통해 어떤 잘못된 입력에도 프로그램이 안정적으로 동작합니다.

4.4 파일 입출력 (saveHighScore / loadHighScore)

게임 종료 시 최고 점수를 파일에 저장하고, 다음 실행 시 불러오는 기능을 구현하였습니다. fopen()의 반환값을 확인하여 파일이 없는 경우에도 오류 없이 동작하도록 처리하였습니다.

```
void saveHighScore(Player p) {
    FILE *fp;          /* 파일 포인터 */
    int oldScore = 0;   /* 기존 최고 점수 */
    char oldName[30] = ""; /* 기존 최고 점수 플레이어 이름 */

    /* 기존 파일에서 최고 점수 읽기 */
    fp = fopen(SAVE_FILE, "r");
    if (fp != NULL) {
        /* 파일이 존재하면 기존 점수를 읽어옵니다 */
        fscanf(fp, "%s %d", oldName, &oldScore);
        fclose(fp);
    }

    /* 이번 점수가 기존보다 높으면 저장 */
    if (p.score > oldScore) {
        fp = fopen(SAVE_FILE, "w");
        if (fp != NULL) {
            fprintf(fp, "%s %d\n", p.name, p.score);
            fclose(fp);
            printf("\n [저장] 새로운 최고 점수를 기록하였습니다! (%d점)\n", p.score);
        }
        else {
            printf("\n [오류] 점수 저장에 실패하였습니다.\n");
        }
    }
    else {
        printf("\n [기록] 현재 최고 점수: %s님의 %d점\n", oldName, oldScore);
        printf("        이번 점수: %d점 (갱신 실패)\n", p.score);
    }
}
```

파일이 존재하지 않으면 fopen()이 NULL을 반환하므로, 이를 먼저 확인하여 최초 실행 시에도 정상 동작하도록 하였습니다. 저장 시에는 이번 점수가 기존 최고 점수보다 높을 때만 파일을 덮어씁니다.

5. 실행 결과

아래는 프로그램을 실제로 실행하였을 때 콘솔에 출력되는 결과입니다. 각 화면은 프로그램의 실행 흐름 순서에 따라 구성하였습니다.

5.1 메인 메뉴 화면

프로그램을 실행하면 가장 먼저 메인 메뉴가 출력됩니다. `showMenu()` 함수가 호출되어 게임 시작, 게임 설명, 최고 점수 확인, 종료 네 가지 선택지가 표시됩니다. 플레이어가 1번을 선택하면 게임이 시작되고, 이름 입력 화면으로 넘어갑니다.

```
=====
[ 무인편의점 관리자 ]
=====
1. 게임 시작
2. 게임 설명
3. 최고 점수 확인
4. 종료
=====
메뉴 선택 : 1

플레이어 이름을 입력하세요 : 박수용

박수용 관리자님, 근무를 시작합니다!
총 5개의 상황을 해결해야 합니다.
```

`initPlayer()` 함수가 호출되어 플레이어 이름을 입력받고, 보안 점수 50, 매출 100, 평판 50, 도난 위험도 30으로 초기값이 설정됩니다.

5.2 게임 진행 화면 (이벤트 발생 예시)

게임이 시작되면 `shuffleEvents()` 함수가 8개의 이벤트를 Fisher-Yates 알고리즘으로 섞고, 앞에서부터 5개를 순서대로 출력합니다. 매 턴 상황이 출력되고 플레이어의 선택을 입력받은 뒤, `showStatus()` 함수가 현재 상태값을 박스 형태로 출력합니다.

```
=====
[ 1번째 상황 / 5턴 ]
=====

[상황] 결제 오류로 인해 손님이 같은 상품을
두 번 결제했습니다.

1. 환불 안내를 한다.
2. 모른 척한다.
3. 다음에 오면 할인해준다고 한다.
선택 (1/2/3): 1

-> 매출은 조금 줄었지만 고객 신뢰가 증가하였습니다.

[ 현재 매장 상태 ] |
+-----+
| 보안 점수   : 50 |
| 매출       : 90 |
| 평판       : 65 |
| 도난 위험도 : 30 |
+-----+
```

2번(경고 방송)을 선택한 결과, 보안 점수가 10 증가하고 도난 위험도가 10 감소하였습니다. 선택 결과는 `handleEvent()` 함수 내부의 `switch-case` 문으로 처리되며, 구조체 포인터를 통해 `Player` 구조체의 값이 직접 변경됩니다.

5.3 잘못된 입력 처리 화면

선택지에 숫자가 아닌 문자를 입력하거나 범위를 벗어난 값을 입력하면, `getChoice()` 함수의 입력 검증 로직이 동작하여 오류 메시지를 출력하고 다시 입력을 요청합니다.

```
=====
[ 무인편의점 관리자 ]
=====

1. 게임 시작
2. 게임 설명
3. 최고 점수 확인
4. 종료
=====

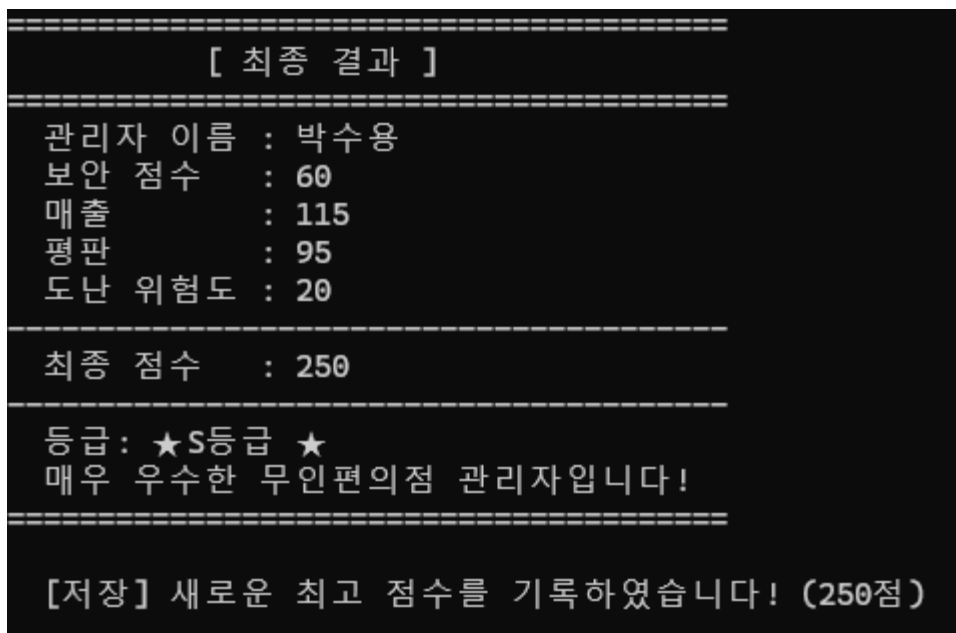
메뉴 선택: 7

[오류] 1~4 사이의 숫자를 입력해주세요.
```

scanf()의 반환값이 1이 아닌 경우 getchar()로 입력 버퍼를 비운 뒤 재입력을 받으므로, 어떤 잘못된 입력에도 프로그램이 비정상 종료되지 않고 안정적으로 동작합니다.

5.4 최종 결과 출력 화면

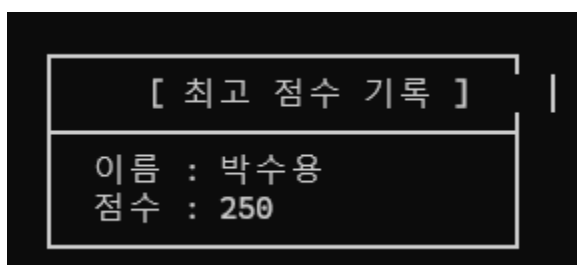
5턴을 모두 완료하면 showResult() 함수가 호출됩니다. 최종 점수는 보안 점수 + 매출 + 평판 - 도난 위험도 공식으로 계산되며, 점수에 따라 S / A / B / C 등급이 출력됩니다.



최종 점수 250점으로 S등급 기준(200점 이상)을 충족하였습니다. saveHighScore() 함수가 기존 highscore.txt의 점수와 비교하여 더 높은 경우에만 파일을 덮어씁니다.

5.5 최고 점수 확인 화면

메인 메뉴에서 3번을 선택하면 loadHighScore() 함수가 highscore.txt 파일을 읽어 저장된 최고 점수를 출력합니다.



6. 어려웠던 점 및 해결 방법

6.1 Fisher-Yates 셔플 구현

처음에는 단순히 `rand() % TOTAL_EVENTS`를 반복하여 이벤트를 선택하려 하였는데, 이 방식은 같은 이벤트가 중복으로 출력될 수 있는 문제가 있었습니다. 5턴 동안 동일한 상황이 반복되면 게임의 재미가 크게 떨어지기 때문에, 중복 없이 랜덤하게 순서를 정하는 방법이 필요하였습니다.

이를 해결하기 위해 Fisher-Yates 셔플 알고리즘을 적용하였습니다. 배열에 0부터 7까지의 인덱스를 순서대로 저장한 뒤, 뒤에서부터 임의의 위치와 교환하는 방식으로 배열 전체를 섞고 앞에서 5개만 꺼내 사용하였습니다. 처음에는 인덱스 범위 계산에서 `rand() % i`와 `rand() % (i+1)`의 차이를 정확히 이해하지 못하여 마지막 원소가 항상 제자리에 남는 버그가 발생하였는데, `i+1`로 수정하면서 해결하였습니다.

6.2 구조체 포인터와 화살표 연산자

함수 내에서 플레이어 상태값을 변경하려고 `Player` 타입을 값으로 전달했더니, 함수가 끝난 후 `main()`의 `player` 변수에 변경 사항이 반영되지 않는 문제가 발생하였습니다. C언어에서 함수에 구조체를 값으로 전달하면 복사본이 전달되기 때문에, 원본을 수정하려면 반드시 포인터를 사용해야 한다는 점을 처음에는 간과하였습니다.

수업 자료를 다시 검토하고 `Player *p`와 같이 구조체 포인터를 매개변수로 받도록 수정하였습니다. 또한 포인터로 구조체 멤버에 접근할 때는 점(.) 연산자가 아닌 화살표(->) 연산자를 사용해야 한다는 점도 이번 프로젝트를 통해 명확히 이해할 수 있었습니다.

6.3 입력 버퍼 처리 문제

사용자가 선택지에서 숫자 대신 문자(예: 'a')를 입력하면, `scanf()`가 값을 읽지 못하고 입력 버퍼에 문자가 남아 있어 이후 `scanf()` 호출이 연속으로 실패하는 무한 루프가 발생하였습니다.

`scanf()`의 반환값이 1이 아닌 경우를 별도의 조건으로 확인하고, `while(getchar() != '\n');` 구문으로 입력 버퍼를 완전히 비운 뒤 다시 입력을 받도록 수정하였습니다. 이를 통해 어떤 잘못된 입력에도 프로그램이 안정적으로 동작하게 되었습니다.

7. 결론 및 느낀 점

이번 기말 프로젝트를 통해 한 학기 동안 배운 C언어 문법을 하나의 완성된 프로그램에 직접 적용해볼 수 있었습니다. 배열, 함수, 구조체, 포인터, 파일 입출력, 반복문, 조건문, 랜덤 함수 등 수업에서 각각 따로 배웠던 개념들이 실제 프로그램 안에서 서로 연결되어 작동하는 것을 확인하면서 각 개념의 필요성과 쓰임새를 더 깊이 이해할 수 있었습니다.

특히 구조체와 포인터를 함께 사용하는 방식이 처음에는 어렵게 느껴졌지만, 직접 오류를 겪고 수정하는 과정을 반복하면서 자연스럽게 익힐 수 있었습니다. 구조체를 값으로 전달할 때와 포인터로 전달할 때의 차이가 실제 프로그램 동작에 어떤 영향을 주는지 체감할 수 있었던 것이 이번 프로젝트에서 가장 인상 깊은 경험이었습니다.

Fisher-Yates 셔플 알고리즘을 도입한 것도 의미 있는 경험이었습니다. 단순히 rand()로 난수를 뽑는 것과 배열 전체를 균등하게 섞는 것의 차이를 직접 코드로 구현하면서 알고리즘의 중요성을 다시 한번 실감하였습니다. 또한 파일 입출력을 통해 게임 종료 후에도 최고 점수가 유지되는 기능을 구현했을 때, 프로그램이 단순한 실행에서 벗어나 상태를 저장하고 관리할 수 있는 수준으로 발전했다는 성취감을 느꼈습니다.

게임 기획 단계에서 보안과 평판의 트레이드오프 구조를 설계한 것도 단순 기술 구현을 넘어, 사용자 경험을 고민하는 과정이었습니다. 앞으로는 더 복잡한 게임 로직과 다양한 자료구조를 활용하여 더욱 완성도 높은 프로그램을 만들어보고 싶다는 목표가 생겼습니다.

8. 참고 자료 및 코드

1. 알고리즘 실습 수업 강의 자료 (배열, 함수, 구조체, 포인터, 파일 입출력, 조건문, 반복문)
2. C언어 rand() 및 srand()를 활용한 랜덤 이벤트 발생 방식 참고
3. Fisher-Yates 셔플 알고리즘 개념 참고
4. 파일 입출력 함수 (fopen, fprintf, fscanf, fclose) 활용 참고
5. 실제 무인편의점 도난·결제 오류·고객 응대 상황 참고
6. 콘솔 기반 선택형 게임 구조 설계 참고

※ 전체 소스 코드: main.c (별도 파일로 제출)